CHELLE Raymond 28/10/2026

Rapport de TP : La remontée des informations du parc

Sommaire :

I. Introduction

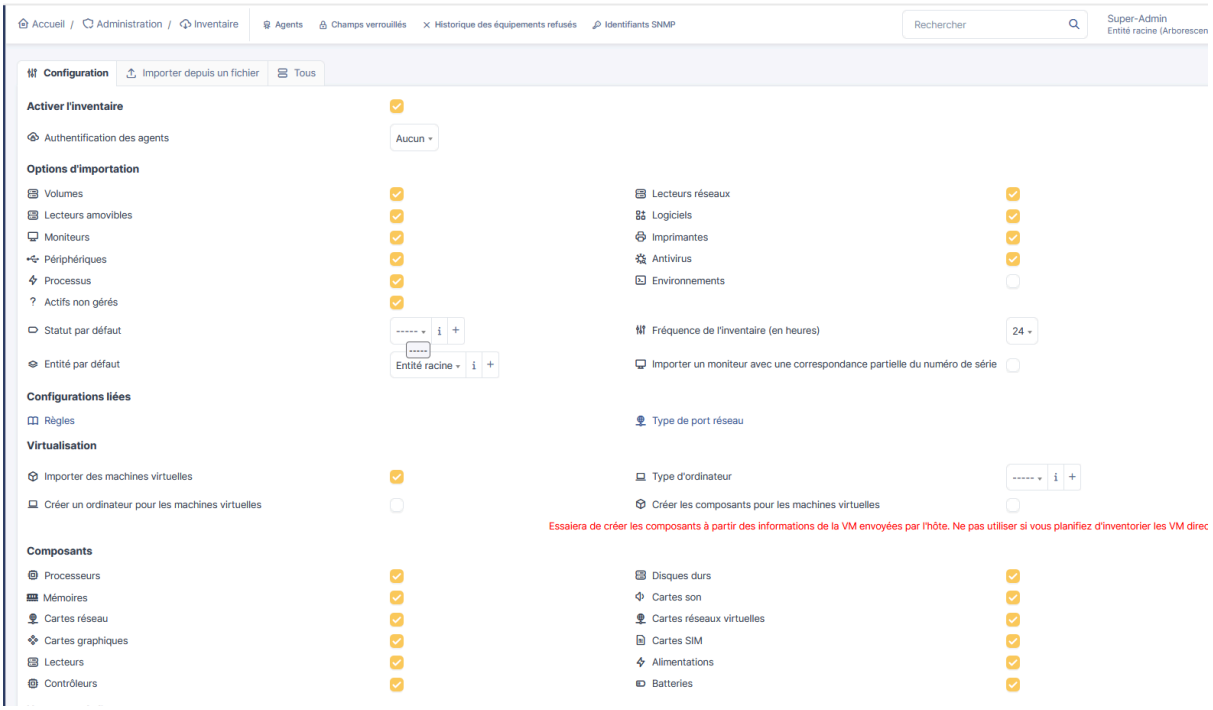
Dans le cadre de la gestion d'un parc informatique d'entreprise, la mise en place d'un inventaire centralisé et automatisé est essentielle pour garantir une visibilité complète sur l'ensemble des actifs matériels et logiciels. Ce travail pratique a pour objectif de déployer et de configurer la solution GLPI afin de recenser un ensemble hétérogène d'équipements au sein d'un même réseau. La démarche adoptée combine l'installation d'agents locaux pour les postes clients, l'interrogation à distance via le protocole SNMP pour les matériels d'interconnexion et les imprimantes réseau, ainsi que la modélisation manuelle pour les périphériques autonomes. Enfin, l'ensemble des mécanismes de découverte et de remontée a été automatisé afin de garantir l'actualisation continue des données sans intervention humaine régulière.

II. Le travail à réaliser

Partie A — Préparation et configuration de l'infrastructure GLPI

Pour initialiser la réception automatisée des inventaires, je me suis d'abord connecté à l'interface d'administration du serveur GLPI (http://172.30.9.43/glpi) avec les privilèges d'administrateur. Je me suis rendu dans les paramètres d'inventaire afin d'activer la fonctionnalité de collecte native côté serveur.

J'ai relevé l'adresse exacte du point d'entrée HTTP d'inventaire : `[http://172.30.9.43/glpi/front/inventory.php](http://172.30.9.43/glpi/front/inventory.php)`.



Avant de passer au déploiement des agents, j'ai vérifié la joignabilité du serveur depuis les machines clientes à l'aide des commandes `ping` et `curl` pour m'assurer que le service Web Apache écoute correctement et qu'aucun pare-feu ne bloque les flux HTTP.

```
C:\Users\ldv>ping 172.30.9.43

Envoi d'une requête 'Ping'  172.30.9.43 avec 32 octets de données :
Réponse de 172.30.9.43 : octets=32 temps=1 ms TTL=64
Réponse de 172.30.9.43 : octets=32 temps<1ms TTL=64
Réponse de 172.30.9.43 : octets=32 temps<1ms TTL=64
Réponse de 172.30.9.43 : octets=32 temps=3 ms TTL=64

Statistiques Ping pour 172.30.9.43:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 0ms, Maximum = 3ms, Moyenne = 1ms

C:\Users\ldv>
```

Partie B — Déploiement de GLPI Agent sous Linux et audit système

Pour auditer finement le poste client sous Linux (srvglpichelle), j'ai procédé au déploiement de l'utilitaire officiel glpi-agent.

Lors de la tentative d'installation initiale par le gestionnaire de paquets de la distribution : **sudo apt install -y glpi-agent**

Le système a retourné l'erreur `E: Impossible de trouver le paquet glpi-agent` en raison de l'absence du paquet dans les dépôts configurés par défaut. J'ai alors tenté d'exécuter le script d'installation unix officiel via la commande : **wget -O-**

https://raw.githubusercontent.com/glpi-project/glpi-agent/develop/contrib/unix/glpi-agent-install.sh | sudo bash

Cette tentative a abouti à une erreur 404 Not Found en transitant par le proxy d'établissement 172.16.63.130:3128.
Pour pallier cette contrainte, j'ai téléchargé directement le paquet binaire Debian de la version stable : **wget https://github.com/glpi-project/glpi-agent/releases/download/1.11/glpi-agent_1.11-1_all.deb**

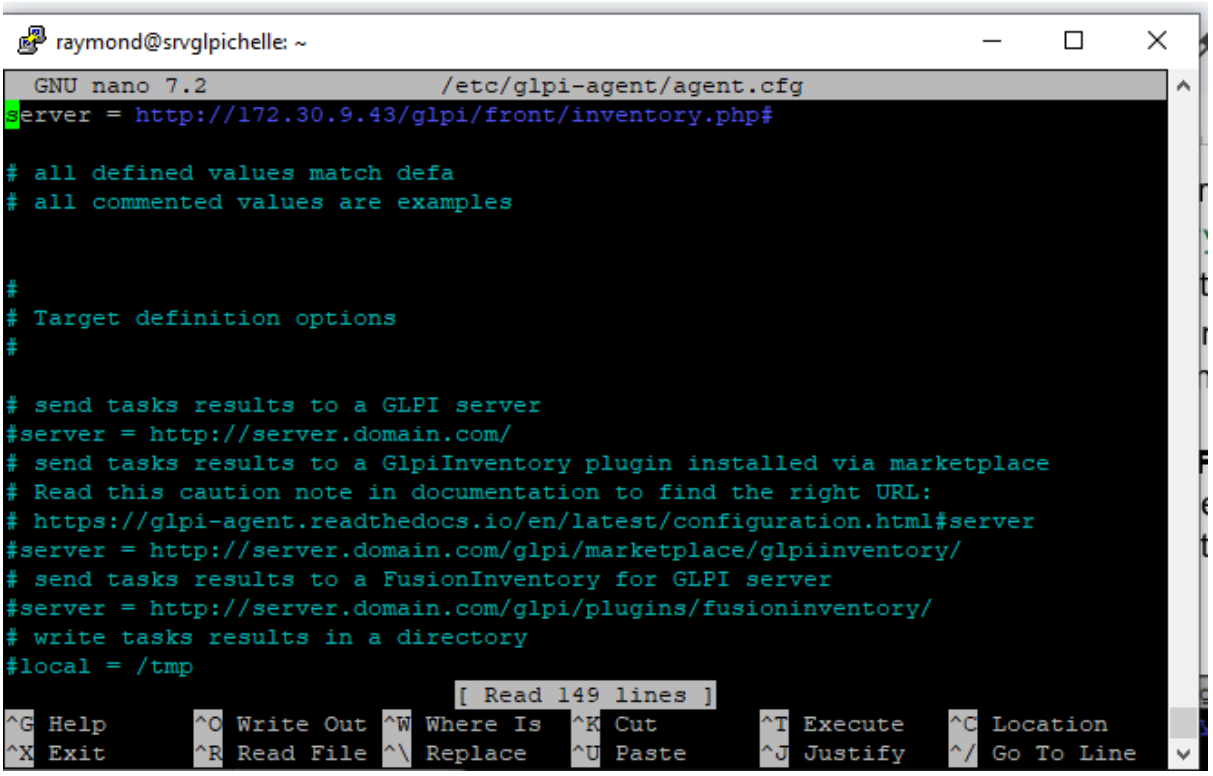
J'ai ensuite lancé l'installation manuelle du paquet : **sudo dpkg -i glpi-agent_1.11-1_all.deb**

L'installation ayant signalé des dépendances non finalisées (`needrestart is being skipped since dpkg has failed`), j'ai assaini la base du gestionnaire de paquets en exécutant successivement : **sudo dpkg --configure -a**
sudo apt install -f -y

J'ai contrôlé la bonne prise en compte du binaire via la commande : **glpi-agent --version**

J'ai ensuite édité le fichier de configuration de l'agent : **sudo nano /etc/glpi-agent/agent.cfg**

J'ai défini l'URL du serveur d'inventaire sur la directive `server` : **server = http://172.30.9.43/glpi/front/inventory.php**



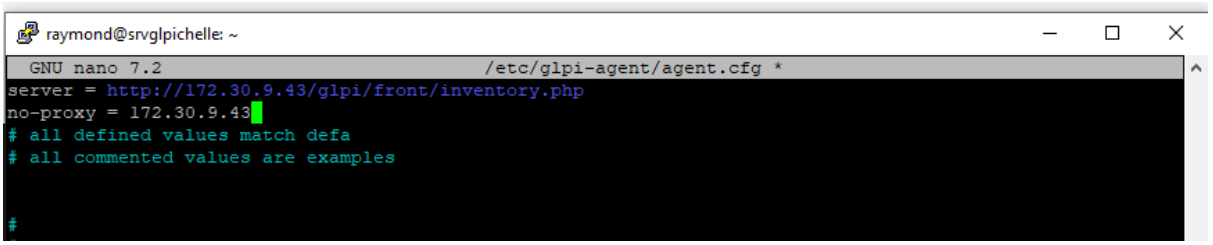
Afin d'assurer une exécution autonome sans exiger l'ouverture d'une session utilisateur, j'ai activé et démarré le service en arrière-plan : **sudo systemctl enable --now glpi-agent**
sudo systemctl status glpi-agent

```
raymond@srvglpichelle:~$ sudo systemctl enable --now glpi-agent
raymond@srvglpichelle:~$
```

J'ai ensuite forcé la première transmission d'inventaire vers le serveur : **sudo glpi-agent --force**
Lors de la première tentative d'inventaire forcé via la commande `sudo glpi-agent --force`, la communication a échoué avec l'erreur `[http client] proxy error: 400 Bad Request`. Deux causes ont été identifiées :

1. La présence d'un caractère parasite `#` à la fin de l'URL cible dans `/etc/glpi-agent/agent.cfg`.
2. Le routage de la requête HTTP locale vers le proxy du réseau pédagogique au lieu d'une communication directe vers le serveur GLPI.

Pour corriger ces dysfonctionnements, j'ai édité `/etc/glpi-agent/agent.cfg` avec `sudo nano /etc/glpi-agent/agent.cfg` afin d'assainir l'URL `server = [http://172.30.9.43/glpi/front/inventory.php](http://172.30.9.43/glpi/front/inventory.php)` et d'ajouter la directive d'exclusion :
no-proxy = 172.30.9.43



J'ai relancé la commande d'inventaire en neutralisant les variables d'environnement du proxy : **sudo no_proxy="172.30.9.43" glpi-agent --force**

```
raymond@srvglpichelle:~$ sudo glpi-agent --force
[sudo] password for raymond:
[info] target server0: server http://172.30.9.43/front/inventory.php
[info] sending contact request to server0
[info] running task Inventory
[info] New inventory from srvglpichelle-2026-10-02-07-53-49 for server0
raymond@srvglpichelle:~$
```

Partie C — Inventaire et caractérisation de l'infrastructure réseau (Passerelle / Routeur)

Dans une infrastructure réseau d'entreprise, les équipements d'interconnexion (switchs, passerelles et routeurs) sont des systèmes embarqués fermés sur lesquels il est impossible

d'installer un agent logiciel résident. Leur recensement doit donc être opéré au travers des protocoles réseau standards (ARP, ICMP, SNMP).

Pour intégrer ces matériels, deux approches complémentaires ont été mises en œuvre : un diagnostic unitaire en ligne de commande pour valider la communication, suivi d'une automatisation complète via le moteur de tâches de GLPI Inventory.

1. Diagnostic de la passerelle et limites SNMP

Pour cartographier le réseau, j'ai d'abord identifié la passerelle par défaut depuis le poste client Linux (`srvglpichelle`) : ip route show

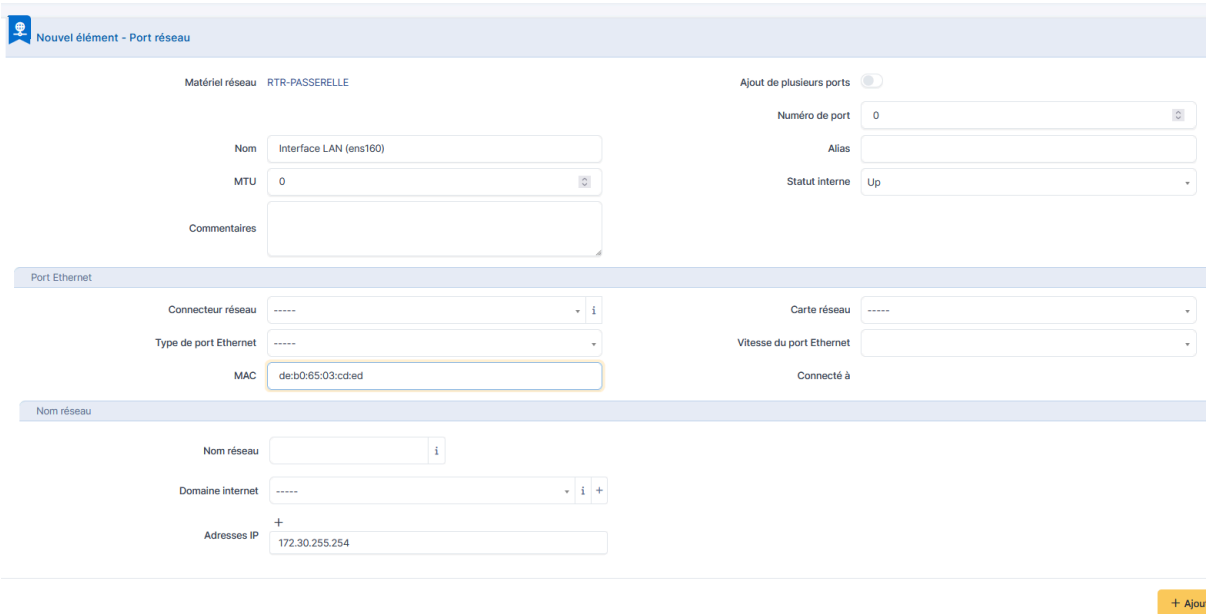
Le résultat indique la passerelle `172.30.255.254` sur l'interface `ens160`.

J'ai ensuite testé la communication SNMP v2c vers cet équipement :

snmpwalk -v 2c -c public 172.30.255.254 1.3.6.1.2.1.1.1.0
La commande a retourné une erreur `Timeout: No Response`. Le service SNMP de la passerelle d'infrastructure est soit désactivé, soit restreint par une liste de contrôle d'accès (ACL).

Pour pallier ce blocage et référencer malgré tout cet actif critique, j'ai extrait son empreinte physique de couche 2 via la table ARP locale :
ip neigh show 172.30.255.254

L'adresse MAC `de:b0:65:03:cd:ed` a ainsi été relevée, me permettant de modéliser manuellement la fiche du routeur `RTR-PASSERELLE` et son port réseau dans la console GLPI.



2. Méthode 1 : Validation unitaire SNMP en ligne de commande (Switch de salle)

Afin de vérifier si d'autres équipements réseau répondaient aux requêtes SNMP standard, j'ai ciblé le switch d'administration de la salle (`172.30.100.80`) :

1. Contrôle de la résolution ARP :
ip neigh show 172.30.100.80
L'adresse MAC physique `44:e4:d9:cc:7c:93` a été confirmée.
2. Interrogation directe de la MIB standard via SNMP v2c :

L'équipement a répondu immédiatement : iso.3.6.1.2.1.1.1.0 = STRING: "SF300-08 8-Port 10/100 Managed Switch"

Cette réponse valide que le commutateur supporte l'interrogation SNMP avec la communauté `public`. Cette méthode en ligne de commande permet de certifier la faisabilité technique avant d'engager un balayage global.

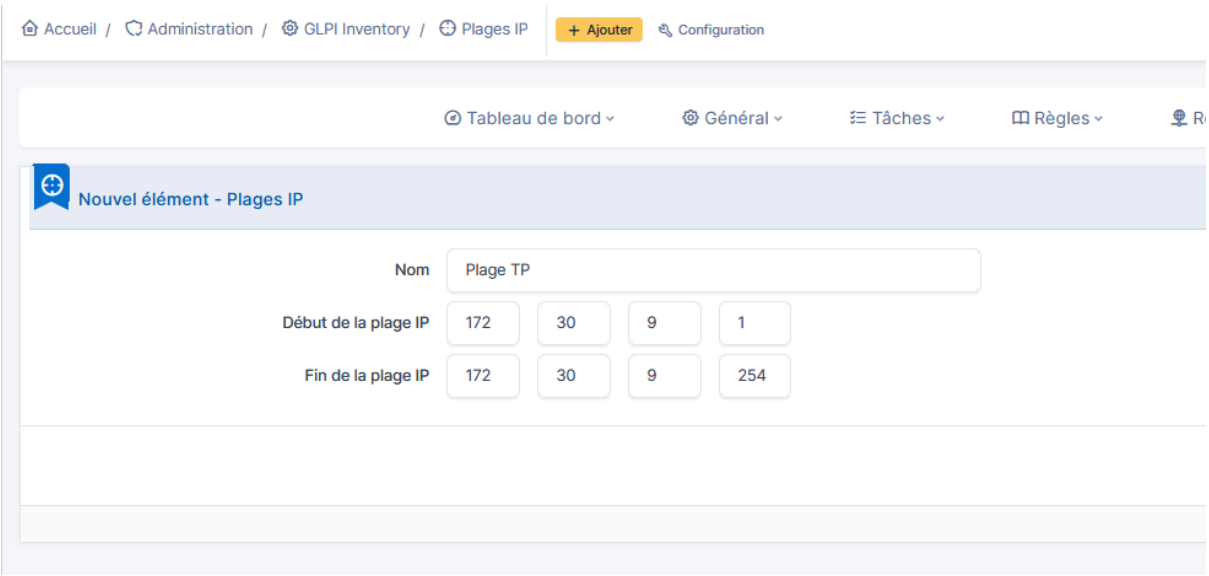
```
raymond@srvglpichelle:~$ ip neigh show 172.30.100.80
172.30.100.80 dev ens160 lladdr 44:e4:d9:cc:7c:93 REACHABLE
raymond@srvglpichelle:~$ snmpwalk -v 2c -c public 172.30.100.80 1.3.6.1.2.1.1.1.0
iso.3.6.1.2.1.1.1.0 = STRING: "SF300-08 8-Port 10/100 Managed Switch"
raymond@srvglpichelle:~$
```

3. Méthode 2 : Découverte et inventaire automatisés via GLPI Inventory

L'interrogation manuelle n'étant pas viable pour un parc important, j'ai configuré la découverte et l'inventaire automatisés directement dans l'interface GLPI, en exploitant l'agent Linux comme sonde d'exécution :

1. Définition de la plage IP cible :

Dans *Administration* > *GLPI Inventory* > *Plages IP*, j'ai déclaré le sous-réseau de travail (du `172.30.9.1` au `172.30.9.254`) sous le nom `Plage TP`.



2. Création de la tâche d'inventaire réseau :

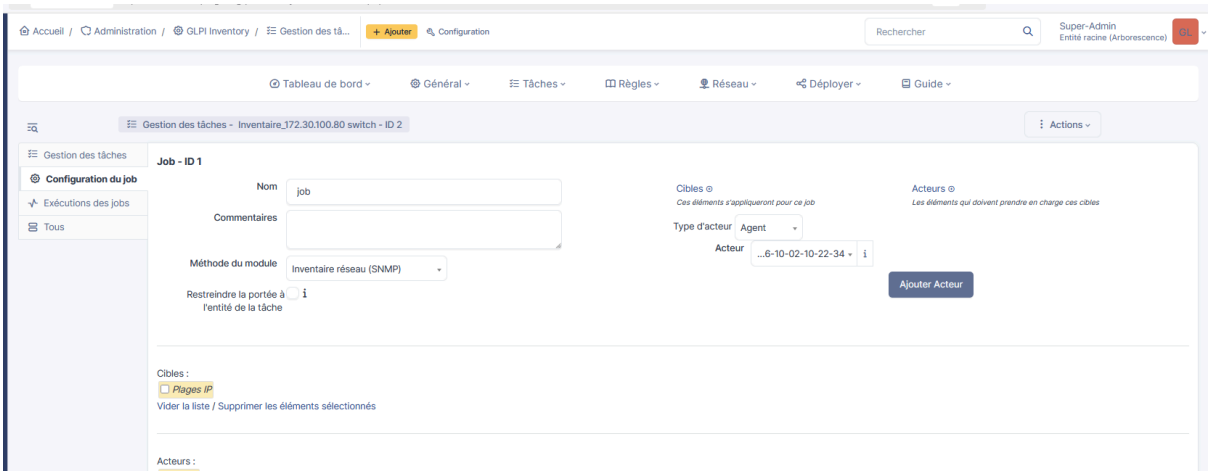
Dans *Administration* > *GLPI Inventory* > *Gestion des tâches*, j'ai créé une nouvelle tâche planifiée (`Job - ID 1`) configurée comme suit

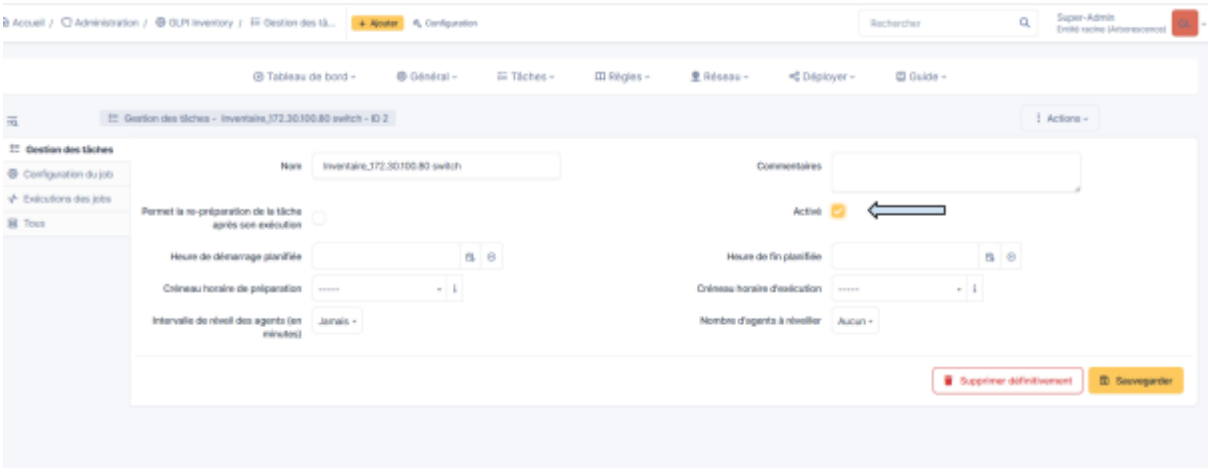
Méthode : *Inventaire réseau (SNMP)*.

Acteur : L'agent GLPI de ma machine Linux (`srvglpichelle`), désigné comme exécutant.

Cible : La plage IP précédemment définie.

Statut : Activée (commutateur « Actif » basculé sur On).





3. Exécution du cycle d'inventaire :

Pour déclencher immédiatement le traitement sans attendre l'ordonnanceur, j'ai forcé la prise de contact de l'agent mandataire vers le serveur : **sudo glpi-agent --force**

L'agent a reçu l'ordre de mission du serveur, scanné les adresses du réseau local via SNMP et téléversé automatiquement les fiches d'inventaire dans GLPI. Le commutateur Cisco

SF300 ainsi que l'ensemble de ses ports et descripteurs sont désormais référencés dans l'inventaire sous *Parc > Matériel réseau.*

Partie D — Intégration et référencement des équipements d'impression

Contrairement aux postes informatiques standards, les équipements d'impression réseau disposent d'un micrologiciel fermé interdisant l'installation directe d'un agent logiciel local. L'inventaire de ces périphériques repose donc sur une interrogation distante par le protocole SNMP opérée depuis le serveur d'administration (172.30.9.43) vers l'imprimante réseau cible (172.30.100.50).

1. Déploiement des modules d'inventaire réseau

Afin de doter l'agent GLPI des capacités de collecte réseau à distance, j'ai installé le paquet additionnel dédié aux tâches d'infrastructure :

sudo apt install -y ./glpi-agent-task-network_1.20-1_all.deb

La vérification de l'arborescence binaire via `ls /usr/bin | grep glpi` a confirmé la disponibilité des exécutables spécialisés `glpi-netdiscovery` et `glpi-netinventory`.

2. Interrogation SNMP distante de l'équipement

Depuis la console du serveur, j'ai lancé l'interrogation SNMP ciblée vers l'imprimante Brother HL-5450DN :

sudo glpi-netinventory --host 172.30.100.50 --credentials version:2c,community:public

L'équipement a répondu favorablement en renvoyant un flux XML exhaustif contenant l'ensemble de ses descripteurs matériels :

- Modèle : Brother HL-5450DN
- Numéro de série : E70647K4N849388
- Compteur global d'impressions : 30 342 pages
- Niveaux de consommables (toner) et adresse MAC physique.

3. Extraction et injection du flux d'inventaire dans GLPI

Pour intégrer ces métriques dans la base de données sans recourir à une saisie manuelle, j'ai redirigé le flux XML collecté dans un fichier d'inventaire local :

sudo glpi-netinventory --host 172.30.100.50 --credentials version:2c,community:public > ~/imprimante.xml

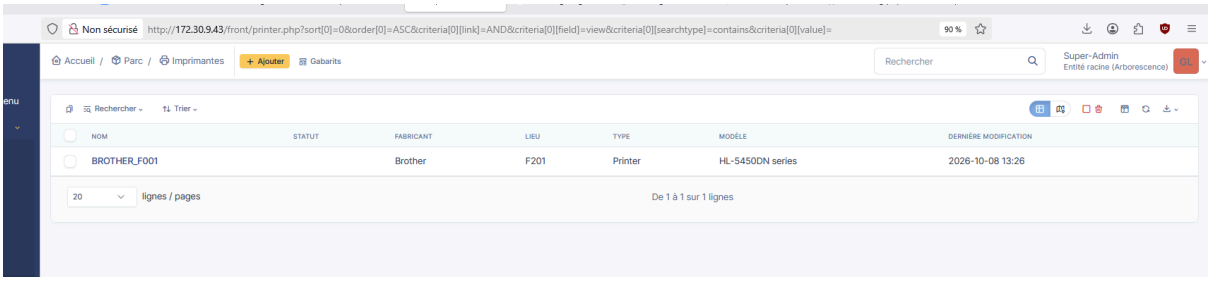
J'ai ensuite poussé ce fichier directement vers le point d'entrée HTTP d'inventaire du serveur GLPI à l'aide de l'utilitaire d'injection :

sudo glpi-injector -v -f ~/imprimante.xml --url http://172.30.9.43/glpi/front/inventory.php

4. Validation et contrôle dans le catalogue GLPI

Après injection, je me suis rendu dans l'interface web sous **Parc > Imprimantes**.

La fiche de l'équipement `BROTHER_F001` a été créée automatiquement avec l'ensemble de ses caractéristiques techniques : modèle certifié, numéro de série matériel, adresse IP de gestion (172.30.100.50) et état des compteurs de pages.

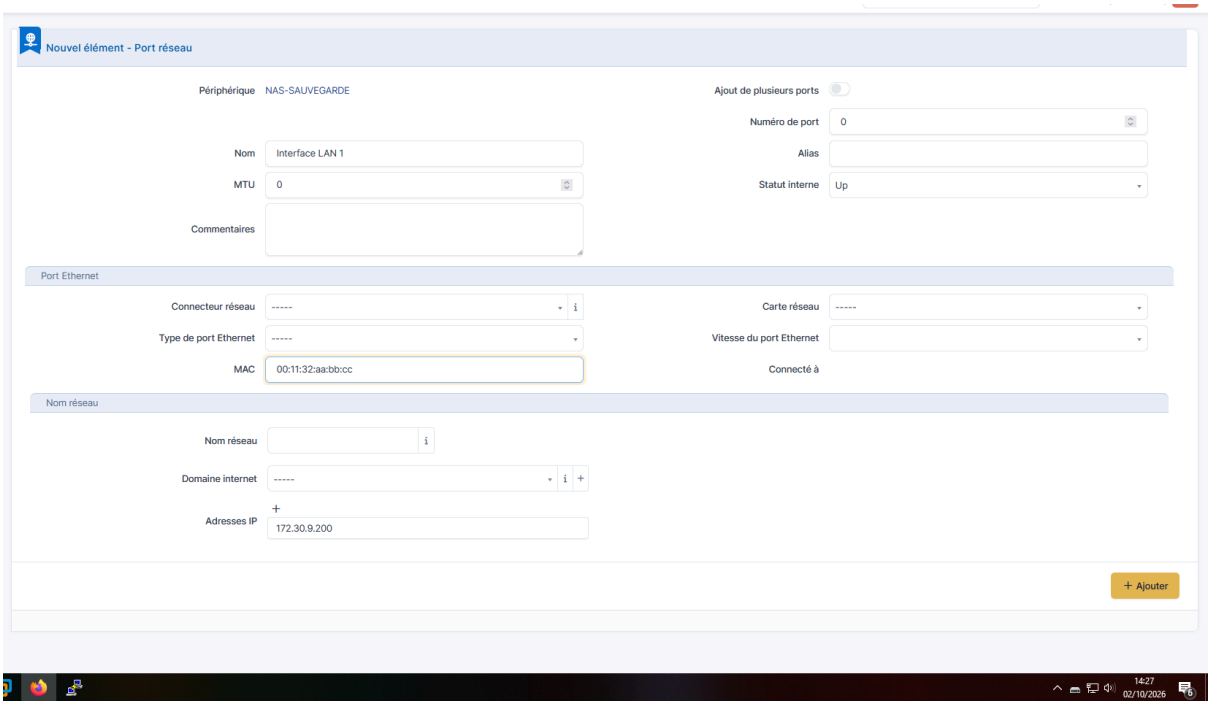


Partie E — Intégration et référencement d'un matériel autonome tiers

Le système d'information comprend des équipements autonomes complémentaires indispensables au fonctionnement de l'infrastructure (serveurs de stockage en réseau NAS, onduleurs, passerelles voix). Ces matériels communicants ne disposent pas d'un système d'exploitation permettant l'exécution d'un agent logiciel standard, mais nécessitent une traçabilité rigoureuse au sein du parc.

Directement depuis la console d'administration de GLPI, j'ai modélisé un serveur de stockage en réseau (NAS) dédié aux sauvegardes :

1. Dans Parc > Périphériques, j'ai créé un nouvel actif sous le nom NAS-SAUVEGARDE, affecté au lieu Salle TP avec le statut actif.
2. J'ai complété les caractéristiques d'exploitation de l'appareil en renseignant le type Stockage ainsi que le constructeur pour fiabiliser le suivi du cycle de vie du matériel.
3. Dans le volet latéral Ports réseau de la fiche, j'ai déclaré son interface physique Ethernet nommée Interface LAN 1 en lui associant son adresse logique fixe (172.30.9.200) ainsi que son adresse physique MAC.



Cette intégration garantit une vision centralisée de l'ensemble des composants physiques de la salle au sein de GLPI, qu'ils soient managés par agent, interrogés par les protocoles réseau ou suivis unitairement comme périphériques autonomes.

Partie F — Automatisation complète des inventaires

Pour répondre aux exigences de production, la mise à jour des inventaires et l'exécution des découvertes réseau doivent s'opérer en continu, sans exiger l'intervention manuelle d'un technicien.

Étape 1 : Autonomie côté client Linux (Démon glpi-agent)

Afin que l'agent logiciel transmette ses rapports de manière périodique dès la mise sous tension de la machine :

sudo systemctl is-enabled glpi-agent

```
raymond@srvglpichelle:~$ sudo systemctl is-enabled glpi-agent
enabled
raymond@srvglpichelle:~$
```

Le retour `enabled` confirme le chargement du service au démarrage système.

Dans le fichier `/etc/glpi-agent/agent.cfg`, la directive suivante cadence les inventaires :

```
raymond@srvglpichelle:~$ sudo nano /etc/glpi-agent/agent.cfg
```

```
raymond@srvglpichelle: ~
  GNU nano 7.2                  /etc/glpi-agent/agent.cfg *
server = http://172.30.9.43/front/inventory.php
delaytime = 86400
```

Cette valeur configure le démon pour collecter et expédier un inventaire complet toutes les 24 heures (86 400 secondes).

Étape 2 : Ordonnancement côté serveur (Cron GLPI)

Afin que GLPI exécute automatiquement les tâches planifiées (inventaires périodiques, découvertes réseau) sans dépendre des visites sur l'interface web, le mode d'exécution des actions automatiques a été basculé sur « CLI ».

Pour déclencher ce moteur chaque minute, j'ai ajouté la ligne suivante dans le fichier /etc/crontab du serveur :

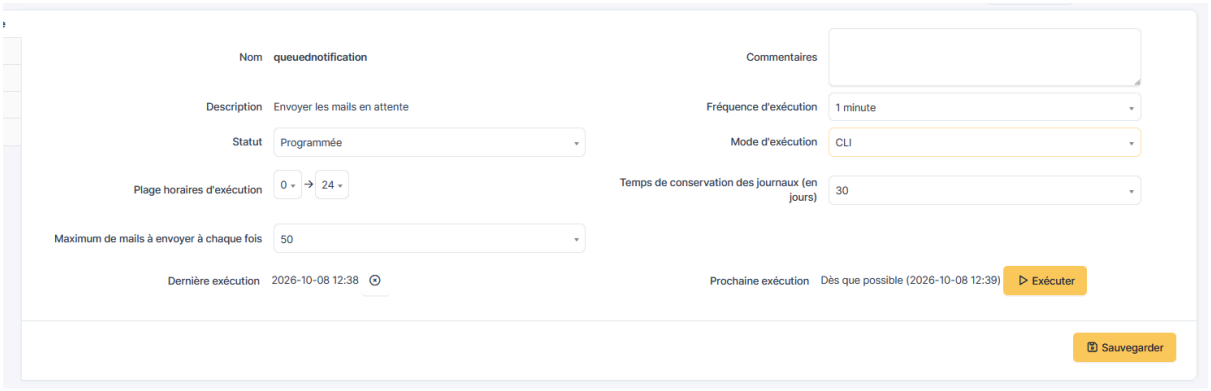
* * * * * www-data php /var/www/glpi/front/cron.php

```
raymond@srvglpichelle: ~
  GNU nano 7.2                          /etc/crontab *
* * * * * www-data php /var/www/glpi/front/cron.php
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from th>
#PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Example of job definition:
# .---------------- minute (0 - 59)
# |  .------------- hour (0 - 23)
# |  |  .---------- day of month (1 - 31)
# |  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
# |  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,>
# |  |  |  |  |
# *  *  *  *  * user-name command to be executed
17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```



III. Conclusion

Ce travail pratique a permis de valider la couverture intégrale du parc informatique en adaptant la méthode de collecte aux contraintes de chaque famille d'équipements. La combinaison du démon local de l'agent GLPI, du moteur de tâches du plugin GLPI Inventory et du planificateur système Cron assure désormais une synchronisation pérenne et autonome du référentiel.

Néanmoins, cette infrastructure présente certaines limites techniques, notamment la dépendance à la disponibilité des services distants comme le protocole SNMP, qui peut être restreint par des règles de sécurité ou des listes de contrôle d'accès sur certains équipements sensibles. Dans une perspective de passage à l'échelle sur un parc de grande envergure, il serait pertinent de déployer des sondes d'inventaire réparties par sous-réseau, de généraliser l'utilisation de protocoles sécurisés comme SNMP v3, et d'automatiser le déploiement des agents clients par des stratégies de groupe (GPO) ou un outil de gestion de configuration centralisé.